Survival-Trajectory Phenotypes from Individualized Survival Curves with unsurv

Validity projection, time-weighted integral distances, and medoid clustering for predicted survival curves.

Imad EL BADISY

2026-07-27

Table of contents

Input representation . . . 1
Validity projection . . . 2
Time-weighted integral distances . . . 3
Medoid clustering . . . 3
Prediction for new patients . . . 5
Stability . . . 5
Example . . . 5
Plots . . . 8
METABRIC application . . . 10
A structural limitation worth carrying forward . . . 16
Interpretation . . . 17

Unsupervised phenotyping usually clusters molecular or clinical covariates and tests survival differences post hoc, which optimizes a covariate representation rather than the time-to-event prediction itself, and can miss structure expressed mainly through when risk accumulates. Many survival learners instead output individualized survival trajectories $\hat{S}_i(t)$ rather than only a scalar risk score. `unsurv` clusters those trajectories directly: the input is a survival probability matrix, not the covariate matrix.[1]

Input representation

Let n be the number of individuals and $t_1 < \cdots < t_Q$ a strictly increasing time grid. For individual i, the observed or predicted curve is $\mathbf{S}_i = (S_i(t_1), \ldots, S_i(t_Q))$, stacked into a matrix

[1] `unsurv` package documentation, CRAN: https://CRAN.R-project.org/package=unsurv.

$S \in [0,1]^{n \times Q}$. A mathematically valid survival curve lives in

$$\mathcal{S}_Q = \{\mathbf{u} \in [0,1]^Q : u_1 \geq u_2 \geq \cdots \geq u_Q\},$$

bounded and non-increasing. The goal is to partition the n trajectories into K clusters by similarity of temporal survival behavior, working directly with S rather than with the covariates that produced it.

Validity projection

Predicted curves from a flexible learner are not guaranteed to land in $\mathcal{S}_Q$: small local irregularities can push values outside $[0,1]$ or briefly break monotonicity. `unsurv` applies a deterministic map $\Pi : \mathbb{R}^Q \to \mathcal{S}_Q$ before computing any distance.

Clamping bounds each value to $[0,1]$,

$$S_i(t_q) \leftarrow \min\{1, \max\{0, S_i(t_q)\}\}.$$

Monotonicity enforcement then sweeps left to right, pulling each point down to no more than the already-corrected previous point,

$$S_i(t_q) \leftarrow \min\left(S_i(t_q), S_i(t_{q-1})\right), \qquad q = 2, \ldots, Q,$$

which guarantees the non-increasing property by construction rather than by post hoc rejection. An optional median smoothing step with odd window width $w \geq 3$ can follow,

$$S_i(t_q) \leftarrow \text{median}\{S_i(t_r) : r \in [q-h, q+h]\}, \qquad h = (w-1)/2,$$

useful when curves come from a learner flexible enough to show small local wiggles that carry no real signal.

Time-weighted integral distances

Distances are built to approximate normalized continuous-time integrals over the follow-up window, not raw pointwise sums. With $\Delta_q = t_{q+1} - t_q$, trapezoidal quadrature weights are

$$w_1 = \tfrac{1}{2}\Delta_1, \qquad w_q = \tfrac{1}{2}(\Delta_{q-1} + \Delta_q)\ (2 \le q \le Q-1), \qquad w_Q = \tfrac{1}{2}\Delta_{Q-1},$$

normalized so that $\sum_q w_q = 1$. For an L_2 distance, curves are mapped into a weighted Euclidean space by $x_{iq} = \sqrt{w_q}\, S_i(t_q)$, so that ordinary squared Euclidean distance recovers the discretized integral,

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = \sum_{q=1}^{Q} w_q (S_i(t_q) - S_j(t_q))^2 \approx \frac{1}{t_Q - t_1} \int_{t_1}^{t_Q} (S_i(t) - S_j(t))^2 \, dt.$$

Taking the square root of the quadrature weight before differencing is what lets a plain Euclidean norm on the transformed matrix stand in for a time integral of a squared term. For an L_1 distance the weight is applied without the square root, since the absolute-value term is linear rather than quadratic, $x_{iq} = w_q\, S_i(t_q)$, giving

$$\|\mathbf{x}_i - \mathbf{x}_j\|_1 = \sum_{q=1}^{Q} w_q \big|S_i(t_q) - S_j(t_q)\big| \approx \frac{1}{t_Q - t_1} \int_{t_1}^{t_Q} \big|S_i(t) - S_j(t)\big| \, dt.$$

Optional per-column standardization can follow the weighting step when early and late follow-up regions sit on very different scales, as can happen with long follow-up.

Medoid clustering

Given the embedded feature matrix, `unsurv` computes a pairwise dissimilarity matrix, Euclidean for L_2 and Manhattan for L_1, and applies partitioning around medoids.[2] For medoid indices $M = \{m_1, \dots, m_K\}$, the fitted partition minimizes

$$\sum_{i=1}^{n} \min_{m \in M} d(\mathbf{x}_i, \mathbf{x}_m).$$

Each cluster prototype under this scheme is an observed curve, not an averaged one, so the reported prototype is always a real, valid patient trajectory, and the method accepts any dissimilarity rather than requiring one compatible with a centroid update.

[2] Kaufman, L., and Rousseeuw, P. J. (1990). *Finding Groups in Data: An Introduction to Cluster Analysis.* Wiley.

Exact minimization of the PAM objective over all $\binom{n}{K}$ possible medoid sets is NP-hard, so `cluster::pam()` solves it by the classical two-phase heuristic. **BUILD** constructs an initial medoid set greedily: the first medoid is the observation minimizing total dissimilarity to all others, and each subsequent medoid is the not-yet-selected point whose addition to the current set produces the largest reduction in the objective, until K medoids are chosen. **SWAP** then iterates: for every (medoid, non-medoid) pair, evaluate the change in the objective from swapping them, and carry out the single best swap found if it improves the objective, repeating until no swap improves it further, a local, not global, optimum in general (BUILD's greedy construction is what keeps the algorithm's typical performance close to the global optimum in practice, without guaranteeing it).

`eps_jitter` (default 0.001) adds independent $\mathcal{N}(0, \texttt{eps_jitter}^2)$ noise to every entry of the embedded feature matrix before computing dissimilarities, purely to break exact ties: two numerically identical or near-identical predicted curves would otherwise create zero or duplicate distances, which can make BUILD's greedy tie-breaking and the silhouette computation below unstable or arbitrary; the jitter is small enough to leave genuine distances essentially unchanged while resolving degeneracies. Setting `eps_jitter = 0` (used in the fully reproducible examples in this note) disables it.

If K is not specified, `unsurv` scans a candidate range $K \in \{2, \dots, K_{\max}\}$, fits PAM at each, and selects the value maximizing mean silhouette width. For point i assigned to cluster C, with $a(i)$ the mean dissimilarity to every other point in C and $b(i)$ the smallest such mean dissimilarity to any other cluster (the "next best" cluster for i),

$$\mathrm{sil}(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1], \qquad \overline{\mathrm{sil}} = \frac{1}{n} \sum_{i=1}^{n} \mathrm{sil}(i),$$

so $\mathrm{sil}(i)$ near 1 means i sits much closer to its own cluster than to the nearest alternative (well clustered), near 0 means it is roughly equidistant from both (on a boundary), and negative means it is on average closer to another cluster than to its own (plausibly mis-clustered); K is chosen to maximize the average of this quantity over all n curves, a pragmatic exploratory default rather than an inferential test for the number of clusters.

```
Require: S, time grid, preprocessing options, distance type, K or K_max
Apply clamping; optionally enforce monotonicity and smoothing
Compute trapezoidal weights
Construct the weighted feature matrix under L1 or L2 mapping
Compute pairwise dissimilarities
if K unspecified: select K by silhouette scanning
Fit PAM; return labels and medoids
```

Prediction for new patients

For a new curve $\mathbf{S}_{\text{new}}$ on the same time grid, `unsurv` applies the same validity map and feature embedding Φ used at training time, then assigns the nearest medoid,

$$\hat{c}(\mathbf{S}_{\text{new}}) = \arg\min_{k} \; d\big(\Phi(\Pi(\mathbf{S}_{\text{new}})),\; \Phi(\Pi(\mathbf{S}_{m_k}))\big).$$

This is well defined precisely because medoids are stored observed curves from training rather than a fitted functional form, so a new patient is assigned without refitting anything.

Stability

Because predicted curves inherit uncertainty from the upstream survival learner, `unsurv` reports stability under resampling, subsampling without replacement or bootstrap with replacement, with optional perturbations such as additive noise on curves followed by monotonicity enforcement, perturbation of the time weights, or feature-space jitter. Stability is summarized by the Adjusted Rand Index between resampled partitions on overlapping individuals, comparing the resampled clustering against the reference partition.

Example

This is the synthetic data-generating process used for the main figure of the `unsurv` application note: three latent risk regimes are simulated, `survdnn` learns individualized survival curves from covariates without ever seeing the latent regime label, and `unsurv` recovers the regimes from those predicted curves alone.[3]

Ninety patients per regime are drawn from cluster-specific Weibull event-time distributions with progressively larger shape and scale parameters across regimes, plus four covariates whose distributions also shift by regime and which perturb each patient's individual Weibull parameters, so the latent structure stays recoverable but is not a deterministic function of the observed covariates alone.

```
library(survival)
library(survdnn)
library(unsurv)
library(ggplot2)
library(patchwork)

set.seed(123)
```

[3] EL BADISY, I. (2026). *unsurv: Clustering Individualized Survival Curves.* Application note, Bioinformatics Advances. Synthetic example and code reproduced from the manuscript's supplementary appendix.

```r
simulate_weibull_time <- function(shape, scale) {
  scale * (-log(stats::runif(length(shape))))^(1 / shape)
}

n_per_cluster <- 90
cluster_specs <- data.frame(
  true_cluster = 1:3,
  shape   = c(1.08, 1.35, 1.70),
  scale   = c(6.0, 11.5, 19.5),
  x1_mean = c(-2.4, 0.0, 2.4),
  x2_mean = c(-1.6, 0.8, 2.8),
  x3_prob = c(0.20, 0.50, 0.80),
  x4_mean = c(2.0, 0.3, -1.2)
)

sim_list <- vector("list", nrow(cluster_specs))
for (i in seq_len(nrow(cluster_specs))) {
  spec <- cluster_specs[i, ]
  x1 <- rnorm(n_per_cluster, spec$x1_mean, 0.55)
  x2 <- rnorm(n_per_cluster, spec$x2_mean, 0.60)
  x3 <- rbinom(n_per_cluster, 1, spec$x3_prob)
  x4 <- rnorm(n_per_cluster, spec$x4_mean, 0.50)

  shape_i <- pmax(0.8, rnorm(n_per_cluster, spec$shape + 0.03 * x3 - 0.02 * x4, 0.07))
  scale_linpred <- 0.20 * x1 + 0.18 * x2 + 0.35 * x3 - 0.12 * x4
  scale_i <- spec$scale * exp(0.10 * scale_linpred + rnorm(n_per_cluster, 0, 0.06))

  event_time <- simulate_weibull_time(shape_i, scale_i)
  censor_time <- runif(n_per_cluster, 10, 28)
  sim_list[[i]] <- data.frame(
    true_cluster = spec$true_cluster,
    time   = pmin(event_time, censor_time),
    status = as.integer(event_time <= censor_time),
    x1 = x1, x2 = x2, x3 = x3, x4 = x4
  )
}
dat <- do.call(rbind, sim_list)
```

A `survdnn` model is fit on a 70 percent training split using only the observed `(time, status)` outcome and the four covariates, never the latent regime label, and used to predict survival curves for the held-out 30 percent on a common 100-point grid up to the 90th percentile of training follow-up.

```r
idx_train <- sample.int(nrow(dat), floor(0.70 * nrow(dat)))
train <- dat[idx_train, ]
test  <- dat[-idx_train, ]

mod <- survdnn(
  Surv(time, status) ~ x1 + x2 + x3 + x4,
  data = train,
  hidden = c(24, 12), activation = "relu",
  dropout = 0, batch_norm = FALSE,
  epochs = 160, lr = 5e-4,
  loss = "aft", optimizer = "adamw",
  optim_args = list(weight_decay = 1e-5),
  .seed = 123, verbose = FALSE
)

t_max <- as.numeric(quantile(train$time, 0.90, na.rm = TRUE))
time_grid <- seq(0.1, t_max, length.out = 100)
S_test <- as.matrix(predict(mod, newdata = test, type = "survival", times = time_grid))
```

`unsurv` then clusters the predicted test-set curves into $K = 3$ groups, with monotonicity enforcement and light median smoothing against small local irregularities in the network output.

```r
fit <- unsurv(
  S = S_test, times = time_grid, K = 3, K_max = 3,
  distance = "L2", enforce_monotone = TRUE, smooth_median_width = 3,
  standardize_cols = FALSE, eps_jitter = 0, seed = 123
)
fit
```

```
unsurv (PAM) fit
  K:3
  distance:L2 silhouette_mean:0.732
  n:81 Q:100
```

Cluster labels from PAM are arbitrary integers, not ordered by risk, so they are relabeled by each medoid's row mean, a simple average-survival-level summary, before comparing against the known latent regime. Recovery is scored with the Adjusted Rand Index, computed directly from the contingency table rather than via `unsurv_compare()` (which compares partitions using observed outcomes, not against a known simulation label).

```r
comb2 <- function(x) ifelse(x < 2, 0, x * (x - 1) / 2)
adjusted_rand_index <- function(labels_true, labels_pred) {
  tab <- table(labels_true, labels_pred)
  n <- sum(tab)
  sum_ij <- sum(comb2(tab)); sum_i <- sum(comb2(rowSums(tab))); sum_j <- sum(comb2(colSums(ta
  expected <- sum_i * sum_j / comb2(n)
  max_index <- 0.5 * (sum_i + sum_j)
  if (max_index == expected) return(1)
  (sum_ij - expected) / (max_index - expected)
}

cluster_order <- order(rowMeans(fit$medoids))
relabel <- setNames(seq_along(cluster_order), cluster_order)
pred_cluster <- unname(relabel[as.character(fit$clusters)])

contingency <- table(True = factor(test$true_cluster, levels = 1:3),
                     Predicted = factor(pred_cluster, levels = 1:3))
contingency
```

```
    Predicted
True  1  2  3
   1 26  0  0
   2  0 26  1
   3  0  0 28
```

```r
accuracy <- sum(diag(contingency)) / sum(contingency)
ari <- adjusted_rand_index(test$true_cluster, pred_cluster)
c(accuracy = round(accuracy, 3), ari = round(ari, 3))
```

```
accuracy      ari
   0.988    0.962
```

Recovery is close to exact: a single class-2 patient is placed in class 3, for a label-matched accuracy of 0.988 and an Adjusted Rand Index of 0.962, computed entirely from curves predicted on held-out patients, with the true regime label never used by either `survdnn` or `unsurv`.

Plots

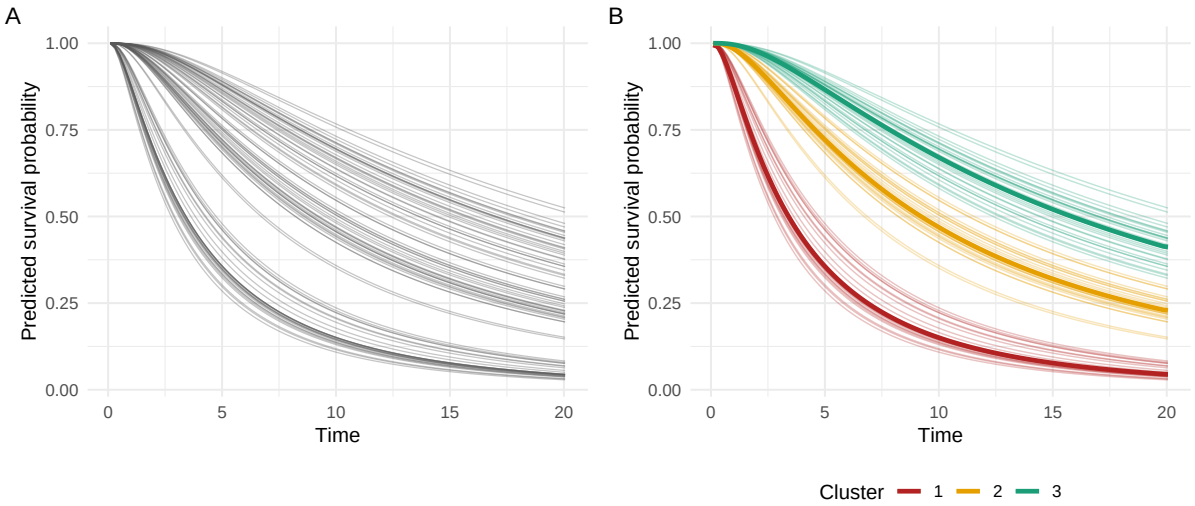
The left panel is the raw model output passed to `unsurv()`, before clustering; the right panel is the same curves colored by the recovered phenotype with medoid prototypes overlaid, produced by `unsurv`'s own `plot()`/`autoplot()` method on the fitted object.

```r
curve_df <- data.frame(
  patient_id = rep(seq_len(nrow(S_test)), each = length(time_grid)),
  time = rep(time_grid, times = nrow(S_test)),
  surv = as.vector(t(S_test)),
  cluster = factor(rep(pred_cluster, each = length(time_grid)), levels = 1:3)
)
S_med <- fit$medoids[cluster_order, , drop = FALSE]
med_df <- data.frame(
  time = rep(time_grid, times = nrow(S_med)),
  surv = as.vector(t(S_med)),
  cluster = factor(rep(seq_len(nrow(S_med)), each = length(time_grid)), levels = 1:3)
)

fig_left <- ggplot(curve_df, aes(time, surv, group = patient_id)) +
  geom_line(linewidth = 0.35, alpha = 0.35, color = "grey35") +
  theme_minimal(base_size = 12) +
  labs(x = "Time", y = "Predicted survival probability") +
  theme(legend.position = "none")

fig_right <- ggplot(curve_df, aes(time, surv, group = patient_id, color = cluster)) +
  geom_line(linewidth = 0.40, alpha = 0.28) +
  geom_line(data = med_df, aes(time, surv, group = cluster, color = cluster), linewidth = 1.3
  theme_minimal(base_size = 12) +
  scale_color_manual(values = c("1" = "#B22222", "2" = "#E69F00", "3" = "#1B9E77")) +
  labs(x = "Time", y = "Predicted survival probability", color = "Cluster") +
  theme(legend.position = "bottom")

fig_left + fig_right + plot_annotation(tag_levels = "A")
```



Panel A alone gives no indication of temporal-risk subgroups; panel B recovers three visually and numerically distinct trajectory groups directly from the predicted curves, without using the observed test-set outcomes at any point, illustrating the intended `unsurv` workflow: an upstream survival learner maps covariates to individualized trajectories, and `unsurv` converts those trajectories into interpretable temporal phenotypes.

METABRIC application

The synthetic example above confirms `unsurv` recovers known structure; this section applies the same pipeline to a real cohort, the METABRIC breast cancer dataset distributed with the `biostatlab` package, and adds two things the synthetic example could not: out-of-sample generalization to a genuinely held-out set, and a comparison against two baseline partitions built on the covariates rather than on predicted curves.[4]

```
library(cluster)

set.seed(20260615)
data(metabric, package = "biostatlab")

raw <- metabric
raw$os_time  <- raw$overall_survival_months
raw$os_event <- raw$overall_survival

exclude <- c("patient_id", "overall_survival_months", "overall_survival", "os_time", "os_even
numeric_cols <- setdiff(names(raw)[vapply(raw, is.numeric, logical(1))], exclude)
```

[4]EL BADISY, I. (2026). *unsurv: Clustering Individualized Survival Curves.* Application note, Bioinformatics Advances. Synthetic example and code reproduced from the manuscript's supplementary appendix.

```r
complete_rate <- vapply(raw[numeric_cols], function(x) mean(!is.na(x)), numeric(1))
numeric_cols <- numeric_cols[complete_rate >= 0.90]
vars <- vapply(raw[numeric_cols], stats::var, numeric(1), na.rm = TRUE)
vars <- vars[is.finite(vars) & vars > 0]
selected <- names(sort(vars, decreasing = TRUE))[seq_len(min(120L, length(vars)))]

model_df <- raw[, c("os_time", "os_event", selected), drop = FALSE]
model_df <- model_df[stats::complete.cases(model_df), , drop = FALSE]
names(model_df) <- make.names(names(model_df), unique = TRUE)
covariate_cols <- setdiff(names(model_df), c("os_time", "os_event"))

nrow(model_df)
```

```
[1] 1839
```

After keeping numeric predictors that are at least 90 percent observed and complete-case filtering, 1839 patients and 120 predictors remain, matching the cohort size reported in the application note. Patients are split three ways: a training set that only **survdnn** sees, a partition set used to fit **unsurv** and the two baselines, and a validation set held out from both, so cluster generalization can be checked directly rather than assumed.

```r
n <- nrow(model_df)
idx_all <- sample.int(n)
n_train <- floor(0.60 * n); n_partition <- floor(0.20 * n)
train      <- model_df[idx_all[seq_len(n_train)], , drop = FALSE]
partition  <- model_df[idx_all[n_train + seq_len(n_partition)], , drop = FALSE]
validation <- model_df[idx_all[(n_train + n_partition + 1):n], , drop = FALSE]

mod <- survdnn(
  Surv(os_time, os_event) ~ .,
  data = train,
  hidden = c(64, 32), activation = "relu",
  dropout = 0.10, batch_norm = TRUE,
  epochs = 120, lr = 5e-4,
  loss = "aft", optimizer = "adamw",
  optim_args = list(weight_decay = 1e-5),
  .seed = 20260615, verbose = FALSE
)

t_max <- as.numeric(quantile(train$os_time, 0.90, na.rm = TRUE))
time_grid <- seq(max(0.1, min(train$os_time[train$os_time > 0], na.rm = TRUE)), t_max, lengt
```

```
pred_partition  <- as.matrix(predict(mod, newdata = partition,  type = "survival", times = ti
pred_validation <- as.matrix(predict(mod, newdata = validation, type = "survival", times = ti

c(train = nrow(train), partition = nrow(partition), validation = nrow(validation))
```

```
     train  partition validation
      1103        367        369
```

The AFT loss is used deliberately, not arbitrarily: as the structural limitation below makes precise, a proportional-hazards learner would make curve-shape clustering here mathematically equivalent to clustering a scalar risk score, which would defeat the purpose of this application.

```
fit <- unsurv(
  S = pred_partition, times = time_grid, K = 3, K_max = 6,
  distance = "L2", enforce_monotone = TRUE, smooth_median_width = 3,
  standardize_cols = FALSE, eps_jitter = 0, seed = 20260615
)
table(fit$clusters)
```

```
  1   2   3
121 102 144
```

```
stab <- unsurv_stability(
  S = pred_partition, times = time_grid, fit = fit, B = 30, frac = 0.70,
  mode = "subsample", jitter_sd = 0.005, weight_perturb = 0.10,
  eps_jitter = 0, return_distribution = TRUE
)
stab$mean
```

```
[1] 0.8488424
```

`unsurv` with $K = 3$ on the partition set gives clusters of size 121, 102, and 144, matching the application note's reported sizes, and resampling stability over 30 subsamples is good (mean ARI 0.849), meaning the partition is not an artifact of one particular sample draw.

Two baselines are fit on the same partition set: PAM on the scalar predicted risk at the median time-grid horizon, and PAM on the first five principal components of the training covariates. Validation-set patients are assigned out-of-sample to all three partitions (`unsurv` via `predict()`, nearest medoid; scalar risk via nearest cluster-mean risk; covariate PCA via

nearest cluster-mean centroid in PCA-score space, with loadings fixed from the training set), and all three are relabeled by partition-set Kaplan-Meier median survival so label 1 is always the worst-survival cluster.

```
relabel_map_by_survival <- function(labels, time, status) {
  old_levels <- sort(unique(as.integer(labels)))
  meds <- vapply(old_levels, function(l) {
    idx <- labels == l
    sf <- survival::survfit(survival::Surv(time[idx], status[idx]) ~ 1)
    m <- summary(sf)$table["median"]
    if (is.na(m)) Inf else as.numeric(m)
  }, numeric(1))
  setNames(seq_along(order(meds)), as.character(old_levels[order(meds)]))
}
apply_map <- function(labels, map) unname(map[as.character(as.integer(labels))])

horizon_idx <- which.min(abs(time_grid - median(time_grid)))
scalar_risk_partition  <- 1 - pred_partition[, horizon_idx]
scalar_risk_validation <- 1 - pred_validation[, horizon_idx]
pam_scalar <- cluster::pam(as.matrix(scalar_risk_partition), k = 3)
scalar_centroids <- vapply(sort(unique(pam_scalar$clustering)), function(k)
  mean(scalar_risk_partition[pam_scalar$clustering == k]), numeric(1))
scalar_labels_validation_raw <- vapply(scalar_risk_validation,
  function(v) which.min(abs(v - scalar_centroids)), integer(1))

x_train <- train[, covariate_cols, drop = FALSE]
x_center <- vapply(x_train, mean, numeric(1)); x_scale <- vapply(x_train, sd, numeric(1)); x
pca <- prcomp(scale(x_train, center = x_center, scale = x_scale), rank. = 5)
project_pca <- function(df) {
  x <- scale(df[, covariate_cols, drop = FALSE], center = x_center, scale = x_scale)
  predict(pca, newdata = x)[, 1:5, drop = FALSE]
}
pca_scores_partition <- project_pca(partition); pca_scores_validation <- project_pca(validati
pam_pca <- cluster::pam(pca_scores_partition, k = 3)
pca_centroids <- t(vapply(sort(unique(pam_pca$clustering)), function(k)
  colMeans(pca_scores_partition[pam_pca$clustering == k, , drop = FALSE]), numeric(5)))
d2 <- outer(rowSums(pca_scores_validation^2), rowSums(pca_centroids^2), "+") - 2 * (pca_score
pca_labels_validation_raw <- max.col(-d2)

map_curve  <- relabel_map_by_survival(fit$clusters, partition$os_time, partition$os_event)
map_scalar <- relabel_map_by_survival(pam_scalar$clustering, partition$os_time, partition$os_
map_pca    <- relabel_map_by_survival(pam_pca$clustering, partition$os_time, partition$os_eve
```

```r
partition_labels <- list(
  `unsurv curve` = apply_map(fit$clusters, map_curve),
  `Scalar risk`  = apply_map(pam_scalar$clustering, map_scalar),
  `Covariate PCA` = apply_map(pam_pca$clustering, map_pca)
)
validation_labels <- list(
  `unsurv curve` = apply_map(predict(fit, pred_validation), map_curve),
  `Scalar risk`  = apply_map(scalar_labels_validation_raw, map_scalar),
  `Covariate PCA` = apply_map(pca_labels_validation_raw, map_pca)
)

cmp_partition  <- unsurv_compare(partition_labels,  partition$os_time,  partition$os_event,
cmp_validation <- unsurv_compare(validation_labels, validation$os_time, validation$os_event,

cmp_partition$summary
```

```
         method K min_size max_size    ari_ref
1  unsurv curve 3      102      144 1.00000000
2   Scalar risk 3       91      145 0.83344758
3 Covariate PCA 3       94      158 0.01781246
```

```r
cmp_validation$summary
```

```
         method K min_size max_size    ari_ref
1  unsurv curve 3       81      163 1.00000000
2   Scalar risk 3       69      163 0.81007077
3 Covariate PCA 3       85      173 0.08603131
```

`unsurv curve` is by definition perfectly self-similar (ARI $= 1$). The scalar-risk baseline is highly concordant with the curve-based partition on both sets (ARI ≥ 0.80), showing that at this single horizon the scalar summary already captures much of the ordering information present in the full curve for this particular model; the covariate-PCA baseline is nearly unrelated to it (ARI ≤ 0.09 on both sets), since it clusters a structurally different representation, the original covariates rather than the model's survival predictions.

```r
rbind(
  cbind(set = "partition",  cmp_partition$cluster_summary),
  cbind(set = "validation", cmp_validation$cluster_summary)
)
```

```
          set cluster   n events median_survival        method
1   partition       1 102     52        166.6667  unsurv curve
2   partition       2 144     68        184.8000  unsurv curve
3   partition       3 121     42        229.3333  unsurv curve
4   partition       1  91     48        164.7333   Scalar risk
5   partition       2 145     66        189.7333   Scalar risk
6   partition       3 131     48        229.3333   Scalar risk
7   partition       1  94     47        177.6333 Covariate PCA
8   partition       2 115     50        193.1333 Covariate PCA
9   partition       3 158     65        205.6000 Covariate PCA
10 validation       1  81     40        159.7333  unsurv curve
11 validation       2 163     63        186.5333  unsurv curve
12 validation       3 125     37        243.1667  unsurv curve
13 validation       1  69     39        152.3000   Scalar risk
14 validation       2 163     60        186.6000   Scalar risk
15 validation       3 137     41        240.2000   Scalar risk
16 validation       1  85     34        167.4333 Covariate PCA
17 validation       2 111     58        176.6000 Covariate PCA
18 validation       3 173     48        240.2000 Covariate PCA
```

For the curve-based partition, median survival increases monotonically with cluster label on both the partition set and the independent validation set, so the ordering "cluster 3 has better survival than clusters 1 and 2" generalizes to patients neither `survdnn` nor `unsurv` ever saw during fitting; the same monotone ordering holds for the scalar-risk baseline. The covariate-PCA baseline's partition-set separation between its two lowest clusters is visibly weaker (a smaller median-survival gap than the other two methods) and only closes on the validation set, consistent with it tracking a different, less risk-aligned signal.

```r
cluster_pal <- c("1" = "#B22222", "2" = "#E69F00", "3" = "#1B9E77")
curve_labels_partition <- apply_map(fit$clusters, map_curve)

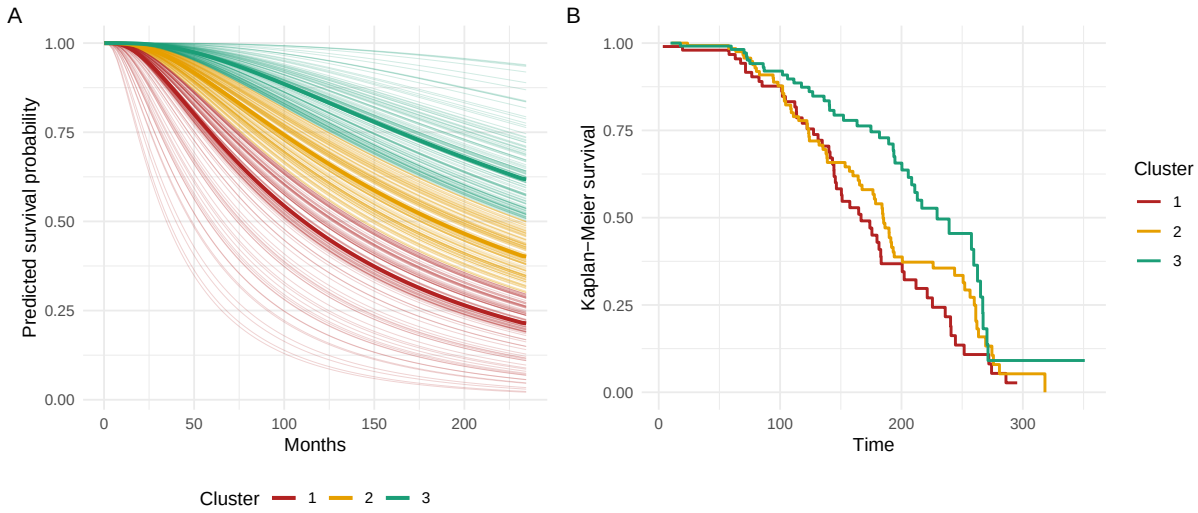
curve_df <- data.frame(
  patient_id = rep(seq_len(nrow(pred_partition)), each = length(time_grid)),
  time = rep(time_grid, times = nrow(pred_partition)),
  survival = as.vector(t(pred_partition)),
  cluster = factor(rep(curve_labels_partition, each = length(time_grid)))
)
medoid_order <- apply_map(seq_len(nrow(fit$medoids)), map_curve)
medoids <- fit$medoids[order(medoid_order), , drop = FALSE]
medoid_df <- data.frame(
  time = rep(time_grid, times = nrow(medoids)),
  survival = as.vector(t(medoids)),
```

```
  cluster = factor(rep(sort(medoid_order), each = length(time_grid)))
)

p_curves <- ggplot(curve_df, aes(time, survival, group = patient_id, color = cluster)) +
  geom_line(linewidth = 0.25, alpha = 0.20) +
  geom_line(data = medoid_df, aes(time, survival, group = cluster, color = cluster), linewid
  scale_color_manual(values = cluster_pal) +
  labs(x = "Months", y = "Predicted survival probability", color = "Cluster") +
  theme_minimal(base_size = 11) + theme(legend.position = "bottom")

p_km <- autoplot(unsurv_compare(list(`unsurv curve` = curve_labels_partition), partition$os_
  scale_color_manual(values = cluster_pal) +
  labs(title = NULL) + theme(strip.text = element_blank())

p_curves + p_km + plot_annotation(tag_levels = "A")
```



Panel A shows `survdnn`-predicted curves on the partition set colored by `unsurv` cluster with medoid prototypes overlaid; panel B is the corresponding Kaplan-Meier curve computed from the *observed* outcomes within each cluster, included to show that the curve-based partition, fit without ever using observed outcomes, recovers groups with genuinely distinct observed survival, not just distinct predicted curves.

A structural limitation worth carrying forward

Under a proportional-hazards model, every predicted curve is $S_i(t) = S_0(t)^{\exp(\eta_i)}$, a scalar power transform of one shared baseline. Any L_1 or L_2 curve-shape distance between two such curves reduces to a monotone function of the scalar linear predictor η_i alone, so clustering

curve shapes under proportional hazards is mathematically equivalent to clustering a single number. `unsurv` is doing genuinely curve-native work only for learners that do not impose proportional hazards, such as random survival forests or a `survdnn` model fit with the AFT or Cox-Time loss, where curve shape carries information beyond one scalar summary.

Interpretation

A practical reporting template is:

- Cluster 1: slow decline in predicted survival, high expected event-free time.
- Cluster 2: intermediate decline, moderate expected event-free time.
- Cluster 3: rapid decline, low expected event-free time.

After clustering, each phenotype should be described with survival probabilities at clinically meaningful horizons, restricted mean survival time summaries, and baseline covariate distributions, so the phenotype stays anchored to survival behavior while remaining interpretable to clinical collaborators. Partitions can be compared against alternative clusterings, such as one built on a scalar risk summary or on covariate principal components, with `unsurv_compare()`, which reports cluster sizes, Adjusted Rand Index agreement, and per-cluster Kaplan-Meier medians rather than a log-rank test, since testing a partition against the labels it was fit to separate is circular.